

Universidade Estadual de Santa Cruz (UESC)

Etapa 1 — Benchmark de Geração do Conjunto de Mandelbrot
(Escape-Time): Implementação sequencial e paralela em memória
compartilhada com OpenMP.

Relatório acadêmico — DEC107 — Processamento Paralelo

Autores: Cibelle Sousa Rodrigues, Lucas Braga Orihuela

Disciplina: DEC107 — Processamento Paralelo

Data: 20/09/2026

1. Introdução

O objetivo principal desta primeira etapa do projeto é avaliar o aprendizado prático dos conceitos de processamento paralelo em memória compartilhada utilizando a API OpenMP. Para cumprir esta meta, o trabalho propõe o desenvolvimento e a otimização de um benchmark focado no Conjunto de Mandelbrot através do método de escape-time. Além disso, esta etapa implementa as versões paralelizáveis do algoritmo, medindo o impacto da variação do número de threads no tempo de execução e investigando estratégias eficazes para contornar o desbalanceamento de carga inerente a este problema matemático.

O Conjunto de Mandelbrot é definido como o conjunto de números complexos c para os quais a sequência matemática iterativa $z_{n+1} = z_n^2 + c$, com $z_n = 0$, permanece limitada. Na implementação computacional, o algoritmo de escape-time verifica cada ponto do plano complexo mapeado, iterando essa fórmula até que o valor de $|z|$ ultrapasse um raio de escape pré-definido, que neste projeto é $|z| > 2$ (ou simplesmente $|z|^2 > 4$). O número de iterações necessárias para que o ponto "escape" deste raio, limitado a um teto máximo ($\text{MAX_ITER} = 1000$), dita a sua coloração na geração da imagem fractal.

Este problema é embaraçosamente paralelo, porém fortemente desbalanceado em relação ao custo por elemento, o que justifica a escolha da geração deste fractal como

benchmark. O custo computacional processado em cada pixel da imagem é diretamente proporcional ao seu escape-time. Pixels localizados fora do conjunto escapam do raio em poucas iterações, exigindo pouco esforço da máquina, enquanto que pixels localizados na fronteira fractal ou no interior do conjunto demandam o limite máximo de iterações, consumindo muito mais processamento. Devido a essa heterogeneidade geométrica da carga computacional, uma divisão mais intuitiva do trabalho resulta em threads ociosas esperando por threads sobrecarregadas, o que torna a análise do desbalanceamento de carga o desafio central desta etapa.

Para lidar com esta questão de desbalanceamento, o projeto fundamenta-se no paradigma de memória compartilhada aliado ao uso da API OpenMP. Diferentemente da execução sequencial por um único processo, o OpenMP distribui o processamento dos laços paralelos entre múltiplas threads através da diretiva *for*. Além disso, a ferramenta oferece a cláusula *schedule*, que permite controlar como as iterações são distribuídas e testar diferentes políticas de escalonamento de laço. Dentre elas, destacam-se:

- **static:** As iterações são divididas em chunks de tamanho fixo e distribuídas às threads em round-robin, ou seja, a thread 0 pega o chunk 0, a thread 1 o chunk 1 e assim por diante voltando ao início. Quando nenhum chunk é especificado, cada thread recebe um único bloco contíguo de $\sim N/p$ iterações. O custo de escalonamento é zero pois tudo é decidido antes da execução, mas o resultado depende fortemente da escolha do chunk. Blocos contíguos grandes concentram trabalho caro numa única thread, enquanto chunks pequenos entrelaçam as iterações e diluem a região custosa entre todas as threads.
- **dynamic:** Mantém uma fila de trabalho sob demanda, portanto, cada thread ao terminar seu chunk solicita o próximo disponível em tempo de execução. Isso adapta a distribuição ao desbalanceamento natural em que threads rápidas processam mais chunks e lentas menos, ao custo de overhead de sincronização por atribuição, que cresce quando o chunk é pequeno.
- **guided:** O tamanho do chunk é decrescente ao longo da execução, proporcional às iterações restantes divididas pelo número de threads. Desta forma, o tamanho do chunk começa grosso, o que significa pouco overhead no início quando o trabalho barato é feito rapidamente, e refina no final, quando a granularidade é fina é também quando sobra a parte cara, buscando um meio termo entre o overhead do dynamic e o desbalanceamento do static.

Por fim, destaca-se que a análise rigorosa de desempenho neste ambiente de memória compartilhada é crucial para verificar a viabilidade do algoritmo paralelo frente ao seu equivalente sequencial. Como o tempo total de execução agora depende não apenas do tamanho do problema, mas do número de processadores e da eficiência na distribuição de trabalho, análises puramente assintóticas tornam-se insuficientes. Assim, avaliar quantitativamente o Fator de Balanço de Carga, além de métricas consolidadas como o Speedup e a Eficiência, é indispensável para diagnosticar e mitigar as

ineficiências provocadas pela heterogeneidade do Conjunto de Mandelbrot.

2. Metodologia

2.1 Ambiente de Execução

Todos os experimentos foram conduzidos em um notebook com sistema operacional Arch Linux (kernel 7.2.6-arch2-1, arquitetura x86_64) e compilador GCC 16.2.1, com as flags `-O3 -std=c11` (versão serial) e `-O3 -std=c11 -fopenmp` (versão paralela). A otimização `-O3` é relevante por se tratar de um benchmark de desempenho, e o `-std=c11` é exigido pelo uso de `clock_gettime` na medição de tempo.

O processador é um Intel Core 7 240H com arquitetura híbrida, um aspecto determinante para os resultados:

- **10 núcleos físicos e 16 CPUs lógicas**, configurando uma combinação de núcleos de performance (P-cores, com Hyper-Threading) e núcleos de eficiência (E-cores, sem Hyper-Threading).
- Caches: L1d 416 KiB, L1i 448 KiB, L2 9,5 MiB, L3 24 MiB (única instância), e 1 nó NUMA.

Essa heterogeneidade de núcleos é central para a interpretação dos resultados: com 16 threads, parte do trabalho roda em núcleos mais rápidos (P) e parte em núcleos mais lentos (E), o que torna o balanceamento de carga mais desafiador e mais revelador do que em uma arquitetura homogênea.

Para controle da frequência, o governador foi fixado em *performance* durante toda a coleta final de dados, por meio do wrapper **`roda_com_performance.sh`**, que liga o modo performance antes dos experimentos e o restaura automaticamente ao final. Essa decisão metodológica é justificada empiricamente pois, na fase inicial (governador em powersave), o baseline serial oscilou em torno de 25% entre sessões (de ~7,6 s para ~5,6 s no input padrão), o que demonstra que a frequência do CPU contamina a medição e inviabiliza speedups confiáveis. Assim, fixar a frequência elimina essa fonte de ruído e torna os números comparáveis e mais confiáveis.

2.1 Implementação Sequencial de Referência

A implementação sequencial base foi desenvolvida na linguagem C e é utilizada tanto como linha de base para o cálculo das métricas de desempenho quanto para a validação da corretude do programa paralelo. O algoritmo sequencial processa a matriz

da imagem linha por linha utilizando laços aninhados simples, onde a função *escape_time* calcula o número de iterações necessárias para cada ponto do plano complexo. Para garantir a precisão dos testes de corretude, a saída do algoritmo gera um arquivo binário contendo a matriz de contagens em inteiros de 32 bits, a qual é comparada byte a byte com o resultado da versão paralela.

2.2 Implementação Paralela com OpenMP

A versão paralela é estruturalmente idêntica à serial, diferindo apenas no núcleo de cálculo. E isto garante que qualquer diferença de resultado (tempo ou corretude) seja atribuível exclusivamente à paralelização, e não a outro fator de implementação.

O paralelismo concentra-se em *gera_mandelbrot_omp*, que usa três diretivas:

- **#pragma omp parallel** — cria o time de threads. Todas as threads executam o bloco e enxergam a mesma matriz, que é alocada uma única vez antes da região, caracterizando o modelo de memória compartilhada.
- **#pragma omp masked** — executado apenas pela thread 0, registra o número real de threads usadas, *omp_get_num_threads()*, que pode ser menor que o máximo solicitado.
- **#pragma omp for schedule(runtime) nowait** — a diretiva central. Distribui as iterações do laço externo entre as threads.

Três escolhas de projeto merecem destaque e justificativa explícita:

- A paralelização é por linha, não por pixel. Cada linha (py) é uma unidade de trabalho indivisível atribuída a uma única thread. Isso reduz o overhead de agendamento, ou seja, há 4096 unidades de trabalho, e não ~16,7 milhões de pixels.
- A política de escalonamento é decidida em tempo de execução, lida da variável de ambiente *OMP_SCHEDULE*. Isso permite testar *static*, *dynamic* e *guided* (e seus chunks) sem recompilar. Desta forma, o mesmo binário serve para todos os cenários. Uma consequência crítica desta escolha, verificada de forma empírica, diz respeito ao comportamento padrão e cabe aqui distinguir dois "defaults" distintos. Quando *OMP_SCHEDULE* não é definida, o *schedule(runtime)* recorre a um escalonamento padrão da implementação, que na *libgomp* utilizada, comportou-se como uma política dinâmica com distribuição desigual de linhas, fator de balanceamento próximo de 1.0 e tempo inferior ao do *static* explícito. Quando *OMP_SCHEDULE* é definida explicitamente como *static* e sem chunk, o OpenMP usa o *static* com particionamento contíguo padrão (blocos de ~N/p iterações por thread), que é justamente o caso de pior desbalanceamento. Ambos foram medidos com propósitos distintos. O primeiro revelou a necessidade de

nunca confiar no default da variável. O segundo serviu de linha de base para quantificar o colapso do static contíguo. Por isso, todos os experimentos oficiais fixam `OMP_SCHEDULE` explicitamente.

- O uso do *nowait* remove a barreira implícita no fim do for. Como cada thread mede o próprio tempo logo em seguida, não há dependência que exija espera coletiva.

Ademais, cada thread mede o seu próprio tempo com *omp_get_wtime()* e conta quantas linhas processou. Os vetores *tempos_thread* e *linhas_thread* são a base do diagnóstico direto de desbalanceamento, ou seja, se uma thread demora muito mais que as outras é a métrica de fator quem denuncia.

2.3 Políticas de Escalonamento Avaliadas

Foram testadas as três políticas de escalonamento do OpenMP, em diferentes granularidades de chunk:

Tabela 1 - Políticas de escalonamento do OpenMP

POLÍTICA	CHUNKS TESTADOS	COMPORTAMENTO ESPERADO
static	default,1,16	Round-robin de chunks fixos; zero overhead; sensível ao tamanho do chunk
dynamic	1,16	Fila sob demanda; adapta-se ao desbalanceamento a custo de overhead
guided	1,32	Chunk decrescente; começa grosso e refina no final

A decisão de incluir chunks de valor 1, ou seja, utilizar granularidade linha a linha em todas as políticas é deliberada e justifica-se na natureza do Conjunto de Mandelbrot. A região de custo máximo é contígua e concentrada, e a granularidade fina é o mecanismo que "diluí" essa região entre as threads. Comparar chunk de valor 1 contra chunk de valor 16/32 permite quantificar o trade-off entre balanceamento e overhead.

2.4 Métricas de Desempenho

Foram adotadas as métricas de desempenho consolidadas na literatura acerca do paralelismo:

- **Tempo de cálculo:** Medido com *clock_gettime* na solução serial e com *omp_get_wtime()* na solução paralela, isolando a parte paralelizável do tempo de I/O.

- **Speedup:** $S(p) = T_{\text{serial}} / T_{\text{paralelo}}(p)$, onde T_{serial} é a média do tempo serial de referência (medida com 7 repetições).
- **Eficiência:** $E(p) = S(p) / p$, quantificando o aproveitamento de cada thread.
- **Fator de Balanceamento de Carga:** $F = \max(\text{tempo_thread}) / \text{média}(\text{tempo_thread})$, calculado a partir dos tempos individuais por thread. $F = 1.0$ indica balanceamento perfeito e conforme este valor fique maior, mais uma thread "segurou" as demais. Esta é uma métrica obrigatória para a análise do caso de desbalanceamento acentuado.

2.5 Planejamento dos Experimentos

Os experimentos cobrem os três blocos obrigatórios do enunciado, além do caso base:

- **Escalabilidade forte** — resolução fixa de 4096×4096 e $\text{MAX_ITER} = 1000$, variando o número de threads em 1, 2, 4, 8, 16, para cada combinação de política $\times$ chunk.
- **Escalabilidade fraca** — trabalho por thread aproximadamente constante, escalando a resolução junto com o número de threads. As resoluções foram 4096^2 , 5793^2 , 8192^2 , 11585^2 , 16384^2 para 1, 2, 4, 8, 16 threads, respectivamente, mantendo $\sim 16,7$ milhões de pixels por thread. Foi usada a política *static* com chunk 16 para isolar o efeito da escalabilidade do efeito do escalonamento.
- **Caso de desbalanceamento acentuado** — região do "vale dos cavalos-marinhos", centrada em -0.743643887 , 0.131825904 com largura $3,0 \times 10^{-3}$ no eixo real e $\text{MAX_ITER} = 5000$, com 4 e 16 threads e todas as políticas/chunks. Esta região concentra grande quantidade de pixels de custo máximo e evidencia os problemas de distribuição de trabalho.

Cada configuração foi executada 5 vezes (versão paralela) e o baseline serial 7 vezes, reportando-se média e desvio padrão. Essa escolha decorre da observação empírica de variância entre execuções mesmo com governador fixado, o que desencorajou a utilização de uma única medida.

2.6 Validação da Corretude

A corretude foi validada comparando bit a bit a matriz de contagens gerada pela versão paralela contra a da versão serial de referência, tanto no input padrão quanto no caso de zoom. Por se tratar de problema determinístico com contagens inteiras e aritmética idêntica por pixel, a expectativa do enunciado é de igualdade exata, admitindo-se, no máximo, divergência em $\leq 0,01\%$ dos pixels de fronteira. Em ambos os casos, o resultado foi IGUAL, satisfazendo o critério de corretude.

2.7 Procedimento de Coleta

O fluxo de coleta deu-se com o governador de frequência fixado em *performance* via *roda_com_performance.sh* conforme abaixo:

1. Medição do baseline serial com *mede_serial.sh*. Saída em: *serial_baselines.txt*.
2. Execução da matriz de experimentos com *run_experimentos.sh*. Saída em: *resultados.csv*.
3. Pós-processamento e geração de gráficos com *pos-processa.py*. Saída em: *resultados_agregados.csv*.

O *resultados.csv* registra em seu cabeçalho as condições do ambiente como sistema operacional, compilador, CPU, número de CPUs lógicas e governador, atendendo à exigência do enunciado de documentar detalhadamente o ambiente de execução.

3. Resultados

Esta seção apresenta os resultados experimentais da Etapa 1, organizados na seguinte ordem: Validação de corretude, Escalabilidade forte, Escalabilidade fraca e Análise do caso de desbalanceamento acentuado. Os números reportados são médias de 5 repetições para a versão paralela e 7 repetições para o baseline serial, com desvio padrão, coletados sob governador de frequência fixado em *performance*. O baseline serial de referência foi de strong = 5,55s e zoom = 2,39s.

3.1 Validação de Corretude

A corretude foi verificada comparando, bit a bit, a matriz de contagens produzida pela versão paralela contra a versão serial de referência, tanto no input padrão quanto no caso de zoom. Em ambos os cenários, a saída foi “IGUAIS” representando igualdade exata, sem nenhum pixel divergente.

Isso é o esperado para um problema determinístico com contagens inteiras e aritmética idêntica por pixel com o mesmo *escape_time*, o mesmo mapeamento pixel para plano, a mesma ordem de operações em ponto flutuante. Não houve, portanto, nenhuma divergência de fronteira a justificar, atendendo integralmente ao critério da Seção 5.5 do enunciado.

Figura 1 - Log da corretude input padrão

```
Projeto-DEC107-PP: bash — Konsole

[cibelle@cibsnotebook Projeto-DEC107-PP]$ ./mandelbrot_serial -o ref
./mandelbrot_openmp -o par
cmp ref.bin par.bin && echo "IGUAIS" || echo "DIFERENTES"
Config: 4096x4096 pixels, MAX_ITER=1000
Regiao: Re em [-2, 1], Im em [-1.5, 1.5]
Saída: ref.bin / ref.pgm
Tempo de calculo (s): 7.739816
Tempo de I/O (s): 0.052005
Tempo total (s): 7.791821
Pixels no conjunto: 2814248 de 16777216 (16.7742%)
Config: 4096x4096 pixels, MAX_ITER=1000, threads(max)=16
Regiao: Re em [-2, 1], Im em [-1.5, 1.5]
Saída: par.bin / par.pgm
Threads usadas: 16
Tempo de calculo (s): 0.632969
Tempo de I/O (s): 0.094460
Tempo total (s): 0.727429
Fator balanceamento: 1.0001 (max/media)
Tempo max/min por thread: 0.632521 / 0.632402
  thread 0: 0.632426 s, 247 linhas
  thread 1: 0.632521 s, 211 linhas
  thread 2: 0.632480 s, 253 linhas
  thread 3: 0.632432 s, 245 linhas
  thread 4: 0.632402 s, 231 linhas
  thread 5: 0.632478 s, 280 linhas
  thread 6: 0.632465 s, 266 linhas
  thread 7: 0.632483 s, 277 linhas
  thread 8: 0.632472 s, 211 linhas
  thread 9: 0.632457 s, 289 linhas
  thread 10: 0.632472 s, 281 linhas
  thread 11: 0.632473 s, 223 linhas
  thread 12: 0.632431 s, 284 linhas
  thread 13: 0.632433 s, 289 linhas
  thread 14: 0.632435 s, 289 linhas
  thread 15: 0.632469 s, 220 linhas
RESULTADO,4096,4096,1000,16,0.632969,0.094460,1.0001
IGUAIS
[cibelle@cibsnotebook Projeto-DEC107-PP]$
```

Comparação bit a bit entre a matriz serial de referência e a matriz paralela input padrão.

Figura 2 - Log da corretude input zoom

```
Projeto-DEC107-PP: bash — Konsole
[cibelle@cibsnotebook Projeto-DEC107-PP]$ ./mandelbrot_serial -x -0.743643887 -y 0.131825904 -l 3.0e-3 -i 5000 -o ref_zoom
./mandelbrot_openmp -x -0.743643887 -y 0.131825904 -l 3.0e-3 -i 5000 -o par_zoom
cmp ref_zoom.bin par_zoom.bin && echo "IGUAIS" || echo "DIFERENTES"
Config: 4096x4096 pixels, MAX_ITER=5000
Regiao: Re em [-0.745143887, -0.742143887], Im em [0.130325904, 0.133325904]
Saída: ref_zoom.bin / ref_zoom.pgm
Tempo de calculo (s): 3.300835
Tempo de I/O (s): 0.090956
Tempo total (s): 3.391791
Pixels no conjunto: 1246 de 16777216 (0.0074%)
Config: 4096x4096 pixels, MAX_ITER=5000, threads(max)=16
Regiao: Re em [-0.745143887, -0.742143887], Im em [0.130325904, 0.133325904]
Saída: par_zoom.bin / par_zoom.pgm
Threads usadas: 16
Tempo de calculo (s): 0.278460
Tempo de I/O (s): 0.078674
Tempo total (s): 0.357134
Fator balanceamento: 1.0037 (max/media)
Tempo max/min por thread: 0.277964 / 0.274607
thread 0: 0.276879 s, 289 linhas
thread 1: 0.276400 s, 214 linhas
thread 2: 0.276541 s, 285 linhas
thread 3: 0.277668 s, 216 linhas
thread 4: 0.277763 s, 284 linhas
thread 5: 0.277659 s, 217 linhas
thread 6: 0.277964 s, 252 linhas
thread 7: 0.276683 s, 288 linhas
thread 8: 0.277544 s, 290 linhas
thread 9: 0.276730 s, 254 linhas
thread 10: 0.277434 s, 212 linhas
thread 11: 0.276835 s, 208 linhas
thread 12: 0.277123 s, 290 linhas
thread 13: 0.276886 s, 280 linhas
thread 14: 0.276373 s, 275 linhas
thread 15: 0.274607 s, 242 linhas
RESULTADO,4096,4096,5000,16,0.278460,0.078674,1.0037
IGUAIS
[cibelle@cibsnotebook Projeto-DEC107-PP]$
```

Comparação bit a bit entre a matriz serial de referência e a matriz paralela input zoom.

3.2 Escalabilidade Forte

O experimento de escalabilidade forte mantém a resolução fixa em 4096×4096 e $\text{MAX_ITER} = 1000$, variando o número de threads em 1, 2, 4, 8 e 16. A métrica central é o tempo de cálculo e derivados dele como o speedup ($S = T_{\text{serial}} / T_{\text{paralelo}}$) e a eficiência ($E = S/p$).

3.2.1 Tempo de Cálculo por Política

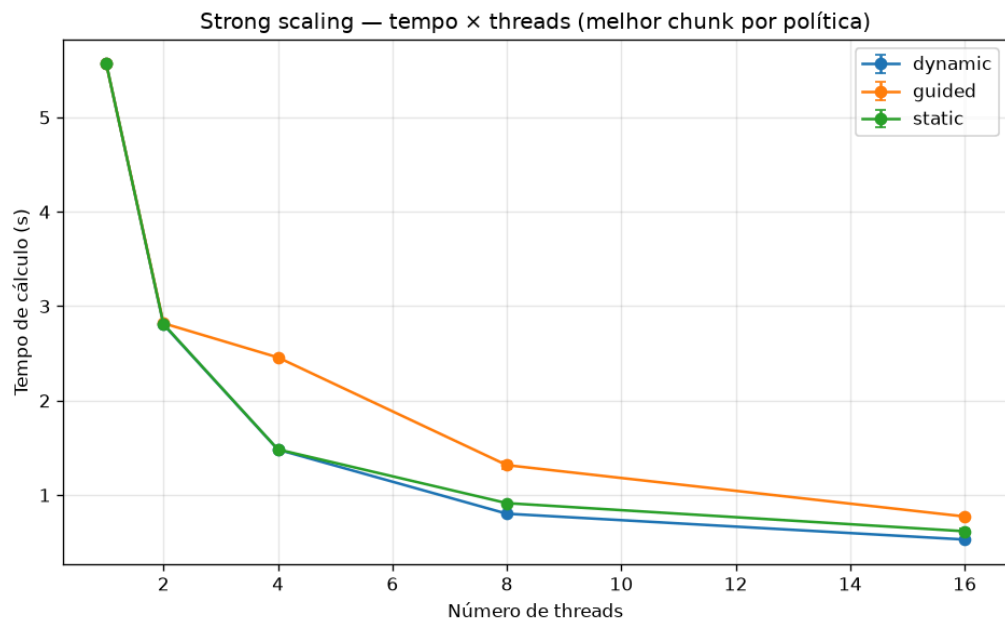
A Tabela 2 resume o tempo médio de cálculo com desvio padrão para a política *static* em seu chunk default, evidenciando o colapso do particionamento ingênuo:

Tabela 2 - Política static com chunk default (NA)

THREADS	TEMPO(S)	FATOR	SPEEDUP	EFICIÊNCIA
1	5,57	1,00	1,00	1,00
2	2,87	1,01	1,93	0,97
4	2,75	1,94	2,02	0,50
8	1,94	2,67	2,85	0,36
16	1,21	3,00	4,57	0,29

O tempo até cai com threads, mas a eficiência despenca de 0,97 com 2 threads para 0,29 com 16. Este é o sintoma clássico de desbalanceamento, onde o ganho vem apenas de "sorte" de uma thread central, não de trabalho bem distribuído.

Gráfico 1 - Tempo de cálculo × número de threads com input padrão



Ao olhar brevemente o Gráfico 1, um aspecto contraintuitivo merece destaque. A curva do *guided* permanece sistematicamente acima da do *static*, mesmo com o melhor chunk de cada um. A justificativa para esse comportamento, ligada à interação entre o chunk inicial “grosso” do *guided* e a distribuição espacial do custo, é discutida em detalhe na Seção 4 deste relatório.

3.2.2 Speedup e Eficiência

A Tabela 3 apresenta, para cada política, o melhor chunk, ou seja, o de menor tempo em 16 threads:

Tabela 3 - Comparativo do speedup e eficiência entre as políticas

POLÍTICA	MELHOR CHUNK	TEMPO(S)	SPEEDUP	EFICIÊNCIA	FATOR
static	1	~0,62	~9,0	~0,56	~1,21
dynamic	1	~0,53	~10,5	~0,66	~1,004
guided	32	~0,77	~7,2	~0,45	~1,60

Conforme vemos na tabela o “*dynamic,1*” é o mais consistente com speedup ~10,5×, fator ~1,004, seguido de “*static,1*” e, por último, o *guided* que ficou atrás até mesmo do “*static,1*”.

Gráfico 2 - Speedup × número de threads com input padrão

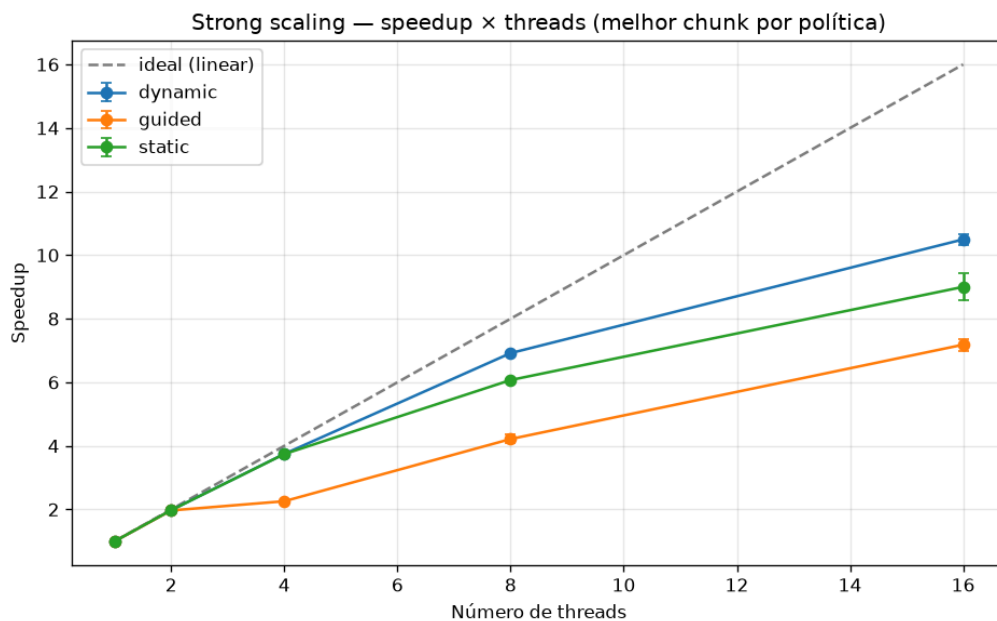
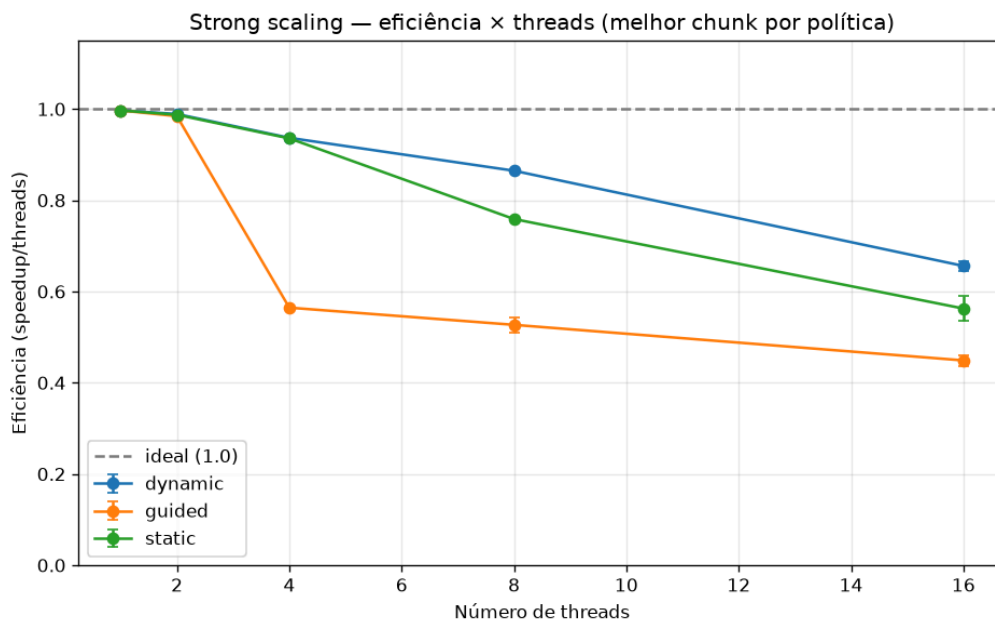


Gráfico 3 - Eficiência × número de threads com input padrão



3.2.3 Fator de balanceamento

Para compreender as diferenças de tempo, speedup e eficiência apresentadas na subseção anterior, é essencial analisar o Fator de Balanceamento de Carga. Esta métrica, calculada pela razão entre o tempo máximo e o tempo médio gasto pelas threads, revela o quão bem o trabalho foi distribuído. Um valor ideal é 1,0, o que indicaria que todas as threads terminaram ao mesmo tempo.

A Tabela 4 compara o fator de balanceamento médio obtido pelas melhores configurações de cada política, contrastando-as com o particionamento ingênuo, utilizando 16 threads no cenário padrão:

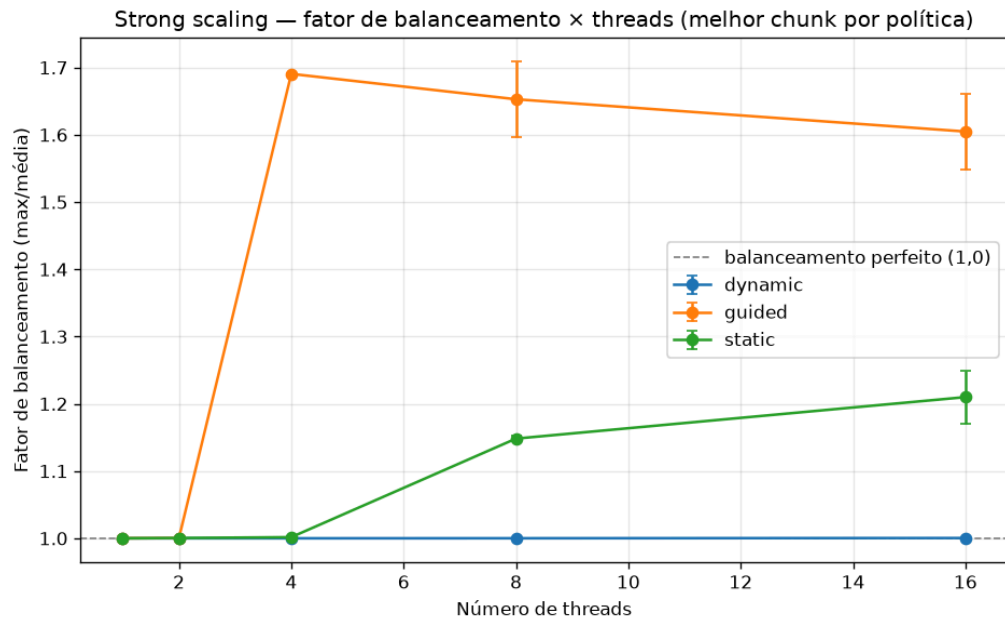
Tabela 4 - Comparativo do Fator de Balanceamento (16 threads)

POLÍTICA	CHUNK	FATOR DE BALANCEAMENTO
dynamic	1	~ 1,004
static	1	~ 1,21
guided	32	~ 1,6
static	default (NA)	~ 3,00

A tabela destaca a disparidade na distribuição de trabalho entre as políticas. Enquanto a configuração “*dynamic,1*” registra um fator médio próximo ao ideal de 1,00, a política

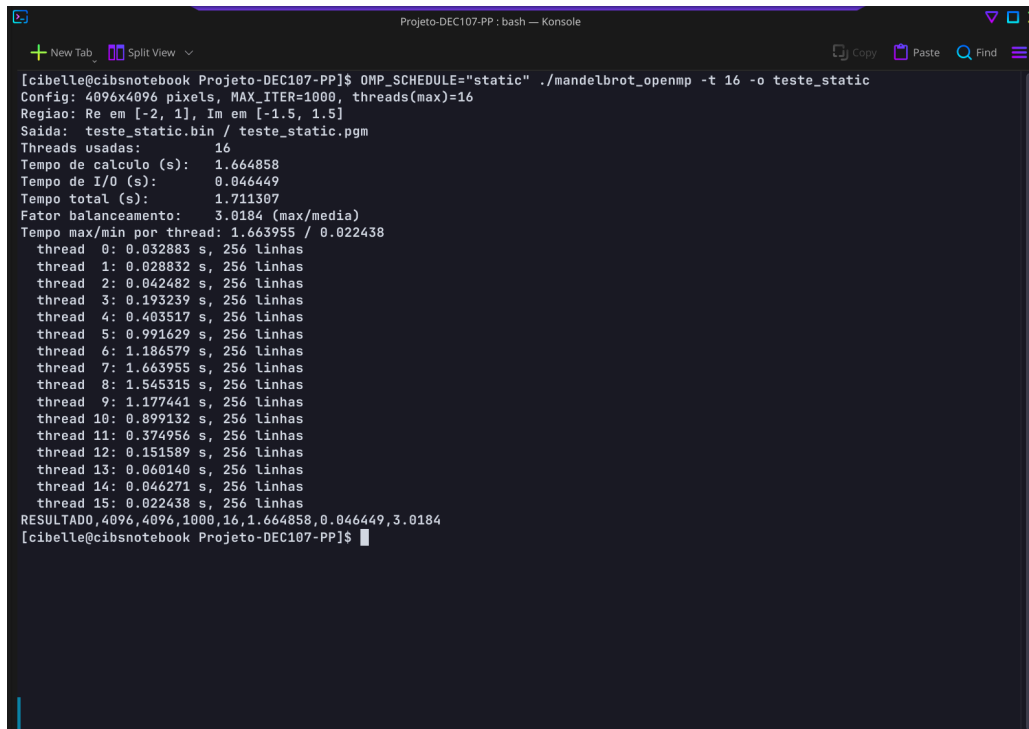
static com particionamento contíguo atinge um fator médio de próximo de 3,00. Essa assimetria levanta questionamentos práticos sobre a alocação das threads no plano complexo, cujas causas geográficas serão exploradas na Seção 4 deste relatório.

Gráfico 4 - Fator de balanceamento das políticas



Para ilustrar de forma qualitativa o comportamento do cenário de maior desbalanceamento, a Figura 3 apresenta o log de execução de um teste manual realizado com o static default em 16 threads:

Figura 3 - Teste manual inicial na fase de validação teórica



```
[cibelle@cibsnotebook Projeto-DEC107-PP]$ OMP_SCHEDULE="static" ./mandelbrot_openmp -t 16 -o teste_static
Config: 4096x4096 pixels, MAX_ITER=1000, threads(max)=16
Regiao: Re em [-2, 1], Im em [-1.5, 1.5]
Saída: teste_static.bin / teste_static.pgm
Threads usadas: 16
Tempo de calculo (s): 1.664858
Tempo de I/O (s): 0.046449
Tempo total (s): 1.711307
Fator balanceamento: 3.0184 (max/media)
Tempo max/min por thread: 1.663955 / 0.022438
thread 0: 0.032883 s, 256 linhas
thread 1: 0.028832 s, 256 linhas
thread 2: 0.042482 s, 256 linhas
thread 3: 0.193239 s, 256 linhas
thread 4: 0.403517 s, 256 linhas
thread 5: 0.991629 s, 256 linhas
thread 6: 1.186579 s, 256 linhas
thread 7: 1.663955 s, 256 linhas
thread 8: 1.545315 s, 256 linhas
thread 9: 1.177441 s, 256 linhas
thread 10: 0.899132 s, 256 linhas
thread 11: 0.374956 s, 256 linhas
thread 12: 0.151589 s, 256 linhas
thread 13: 0.060140 s, 256 linhas
thread 14: 0.046271 s, 256 linhas
thread 15: 0.022438 s, 256 linhas
RESULTADO,4096,4096,1000,16,1.664858,0.046449,3.0184
[cibelle@cibsnotebook Projeto-DEC107-PP]$
```

Ilustra, com o detalhamento por thread, como o desbalanceamento acontece.

O log de saída detalha o tempo exato de execução e a quantidade de linhas da imagem atribuídas a cada uma das 16 threads. Nota-se que threads das extremidades concluíram suas iterações entre em aproximadamente 0,02s a 0,04s, ao passo que as threads centrais demandaram significativamente mais tempo, a exemplo da thread 7, que encerrou seu processamento em 1,66s.

3.3 Escalabilidade Fraca

Na escalabilidade fraca, a resolução cresce junto com o número de threads, mantendo o trabalho por thread aproximadamente constante (~16,78 M pixels/thread). A resolução cresceu de forma que: $4096^2 \rightarrow 5793^2 \rightarrow 8192^2 \rightarrow 11585^2 \rightarrow 16384^2$ para $1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16$ threads. Para isolar o efeito de escalabilidade a política ficou fixada em static,16.

Gráfico 5 - Tempo de cálculo × número de threads

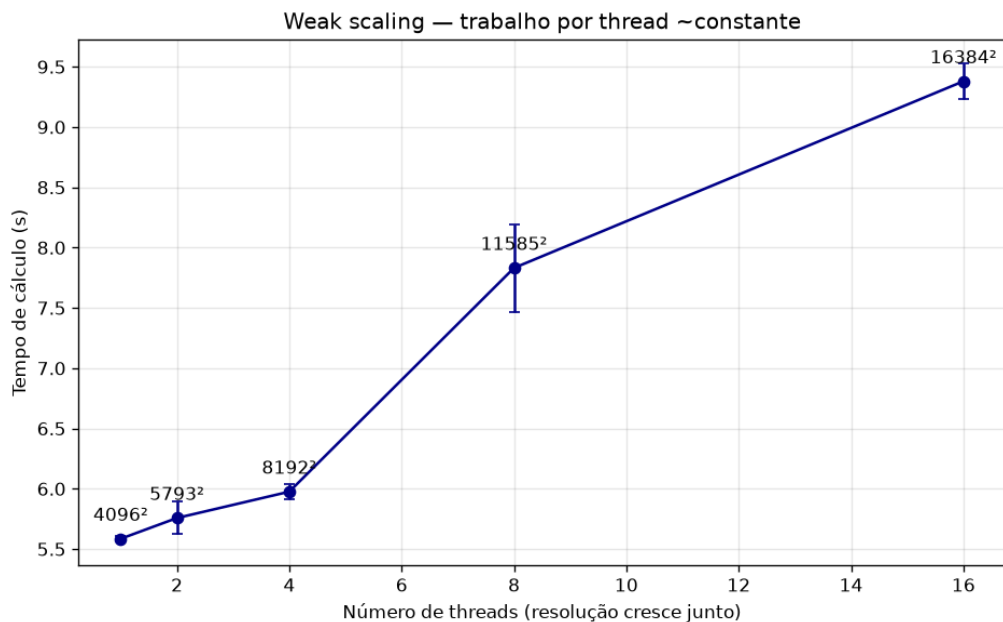


Tabela 5 - Número de threads × resolução × tempo em média

THREADS	RESOLUÇÃO	TEMPO(S)
1	4096 ²	5,59
2	5793 ²	5,76
4	8192 ²	5,98
8	11585 ²	7,83
16	16384 ²	9,38

Conforme podemos ver, o crescimento não-linear a partir de 8 threads evidencia que, com mais threads, o custo deixa de ser dominado pelo cálculo e passa a ser limitado por recursos compartilhados como largura de banda de memória, cache L3 de 24 MiB e a entrada em operação dos E-cores mais lentos. A análise detalhada fica para a Seção 4 deste relatório.

3.4 Caso De Desbalanceamento Acentuado

Esta seção trata do cenário de estresse que usa a região do "vale dos cavalos-marinhos" centrada em $(-0.743643887, 0.131825904)$ com largura real de $3,0 \times 10^{-3}$ e $\text{MAX_ITER} = 5000$. Esta é uma região onde a concentração de pixels de

custo máximo é extrema. E por isso, a análise do Fator de Balanceamento de Carga é obrigatória e crucial para a avaliação dos resultados obtidos com a solução paralela.

Gráfico 6 - Balanceamento de carga (vale dos cavalos-marinhos)

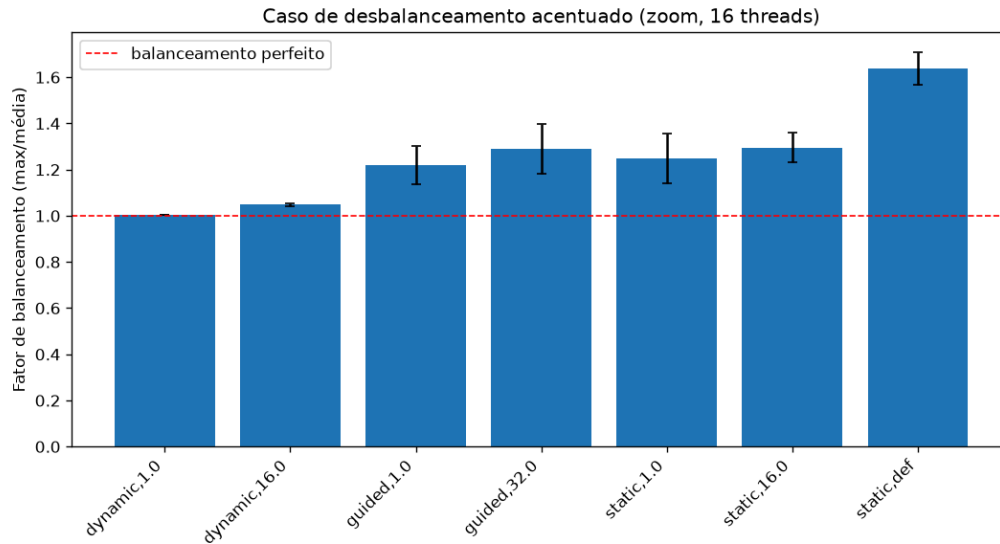


Tabela 6 - Tempo (média) × fator de desbalanceamento por política e chunk

POLÍTICA	CHUNK	TEMPO	FATOR
static	default	0,36	1,64
static	16	0,31	1,30
static	1	0,30	1,25
dynamic	16	0,26	1,05
dynamic	1	0,26	1,004
guided	32	0,31	1,29
guided	1	0,29	1,22

Conforme a Tabela 6 e o Gráfico 6 podemos ver que o “*dynamic,1*” atinge o menor tempo (0,32 s) e o melhor balanceamento (fator 1,004), confirmando que no desbalanceamento acentuado a granularidade linha-a-linha combinada com a adaptação imediata do *dynamic* é a melhor solução.

Figura 4 - Teste manual do dynamic,1 (vale dos cavalos-marinhos)

```
[cibelle@cibsnotebook Projeto-DEC107-PP]$ OMP_SCHEDULE="dynamic,1" ./mandelbrot_openmp -x -0.743643887 -y 0.131825904 -l 3.0e-3 -i 5000 -t 16 -o teste_zoom_dynamic
Config: 4096x4096 pixels, MAX_ITER=5000, threads(max)=16
Regiao: Re em [-0.745143887, -0.742143887], Im em [0.130325904, 0.133325904]
Saída: teste_zoom_dynamic.bin / teste_zoom_dynamic.pgm
Threads usadas: 16
Tempo de calculo (s): 0.335183
Tempo de I/O (s): 0.104119
Tempo total (s): 0.439301
Fator balanceamento: 1.0031 (max/media)
Tempo max/min por thread: 0.334198 / 0.331764
thread 0: 0.332757 s, 267 linhas
thread 1: 0.334162 s, 227 linhas
thread 2: 0.334132 s, 230 linhas
thread 3: 0.334198 s, 230 linhas
thread 4: 0.332461 s, 269 linhas
thread 5: 0.333507 s, 270 linhas
thread 6: 0.332568 s, 263 linhas
thread 7: 0.333966 s, 255 linhas
thread 8: 0.332286 s, 268 linhas
thread 9: 0.334019 s, 272 linhas
thread 10: 0.333843 s, 272 linhas
thread 11: 0.332723 s, 271 linhas
thread 12: 0.331764 s, 272 linhas
thread 13: 0.332515 s, 271 linhas
thread 14: 0.333688 s, 272 linhas
thread 15: 0.332166 s, 187 linhas
RESULTADO,4096,4096,5000,16,0.335183,0.104119,1.0031
[cibelle@cibsnotebook Projeto-DEC107-PP]$
```

Ilustra, com o detalhamento por thread, como o balanceamento é quase ideal.

A Figura 4 reforça, ao nível de thread, que o *dynamic* conseguiu equilíbrio quase perfeito pois todas as threads terminaram praticamente juntas, mesmo processando quantidades de linhas ligeiramente diferentes.

3.5 Síntese Dos Resultados

A tabela abaixo consolida os três achados centrais que irão sustentar a discussão dos resultados presente na seção 4 deste relatório:

Tabela 7 - Síntese dos resultados obtidos

CENÁRIO (16 threads)	MELHOR TEMPO	PIOR TEMPO	PIOR CONFIGURAÇÃO
input padrão	dynamic,1	guided	static default
zoom (vale dos cavalos-marinhos)	dynamic,1	guided	static default

Comparando cada política pelo seu melhor chunk (Gráficos 1–3), o guided é a pior política por ser a mais lenta das três. Mas considerando todas as configurações individuais, o static com chunk default (contínuo) é a pior de todas com fator ~2,99 e

speedup de apenas $\sim 4,6\times$ no input padrão, pior que qualquer configuração do guided (fator $\sim 1,5$).

Em síntese, os resultados convergem para três conclusões centrais. Primeiro, a corretude foi preservada integralmente, a saída paralela é bit a bit idêntica à serial de referência tanto no input padrão quanto no caso de zoom, atendendo ao critério da Seção 5.5 do enunciado. Segundo, a política *dynamic* com chunk unitário mostrou-se a mais consistente nos dois cenários analisados atingindo speedup de $\sim 10,5\times$ no input padrão e $\sim 9,1\times$ no zoom, com fator de balanceamento próximo do ideal ($\approx 1,004$ no zoom), ao passo que o *static* com particionamento contíguo representou o pior caso, com fator $\sim 3,00$ e eficiência em torno de 29% com 16 threads. Terceiro, o escalonamento fraco evidenciou três regimes distintos de comportamento, com degradação acentuada a partir de 8 threads, e o desempenho global ficou aquém do ideal linear, refletindo a natureza híbrida da arquitetura (10 núcleos físicos / 16 CPUs lógicas). Esses três eixos — corretude, *dynamic*,¹ mais consistente que as demais políticas e o limite imposto pela arquitetura — constituem a base para a discussão apresentada na Seção 4.

4. Discussão

Esta seção interpreta os resultados da Seção 3 à luz dos conceitos teóricos da Introdução e das decisões metodológicas registradas na Seção 2. A discussão está organizada em torno de sete eixos: corretude, a comparação de ganhos, o impacto da arquitetura híbrida, o balanceamento de carga de cada política, as escalabilidades, e por fim, os demais gargalos e sugestões de melhorias.

4.1 Análise Da Corretude

A corretude foi validada por comparação bit a bit (cmp) entre a matriz de contagens da versão paralela e a da versão serial de referência, tanto no input padrão quanto no caso de zoom. Em ambos os cenários o resultado foi “IGUAIS”, ou seja, igualdade exata, sem nenhum pixel divergente.

Esse desfecho é exatamente o esperado para este problema. O Mandelbrot por escape-time é um caso conjunto embaraçosamente paralelo: cada pixel é computado de forma independente, e a função *escape_time* executa exatamente a mesma sequência de operações em ponto flutuante, independentemente de qual thread a processa. Como a atribuição de trabalho às threads não altera a ordem das operações dentro de cada pixel, não há reassociação de ponto flutuante capaz de introduzir divergência de fronteira. A igualdade exata observada é, portanto, a consequência estrutural do desenho da paralelização, atendendo integralmente ao critério da Seção

5.5 do enunciado, que admite divergência apenas em situações de reassociação, as quais não ocorrem aqui.

4.2 Comparação Entre a Versão Sequencial e a Versão Paralela

A corretude foi validada por comparação bit a bit (cmp) entre a matriz de contagens da versão paralela e a da versão serial de referência, tanto no input padrão quanto no caso de zoom. Em ambos os cenários o resultado foi “IGUAIS”, ou seja, igualdade exata, sem nenhum pixel divergente.

O ganho proporcionado pela paralelização é substancial e, mais importante, é fortemente condicionado pela política de escalonamento. Com 16 threads e o baseline serial de referência de 5,55s no input padrão, os resultados variam de um speedup modesto de $\sim 4,6\times$, utilizando *static* contíguo, até $\sim 10,5\times$ utilizando “*dynamic, 1*”. Em outras palavras, para o mesmo problema e o mesmo ambiente de execução, a escolha correta do escalonamento quase dobra o ganho de tempo.

Esse contraste é crucial para uma lição desta etapa. Em um problema fortemente desbalanceado, a paralelização por si só não garante desempenho, pois, a forma como o trabalho é distribuído é fator extremamente importante. A versão paralela é estruturalmente idêntica à serial, contendo o mesmo *escape_time* e mesmo mapeamento pixel→plano, o que significa que toda a diferença de desempenho entre as configurações decorre exclusivamente da política de agendamento, o que torna o experimento um ótimo caso de estudo de balanceamento de carga.

4.3 Por Que o Problema é Desbalanceado e a Resposta de cada Política

O Conjunto de Mandelbrot é caracterizado por ser um problema embaraçosamente paralelo, o que significa que não há dependências de dados entre as threads para o cálculo de diferentes pixels. Contudo, essa característica oculta a severa heterogeneidade geométrica da carga computacional que é um desafio central para sistemas de alto desempenho. Em outras palavras, o tempo necessário para processar cada ponto é proporcional ao número de iterações do laço de escape e enquanto a vasta maioria dos pixels situados em regiões afastadas do fractal escapam rapidamente do raio pré-definido (exigindo pouco processamento), os pixels no interior ou na fronteira do conjunto demandam as 1.000 iterações completas no input padrão, e até 5.000 iterações como no caso de zoom no vale dos cavalos-marinheiros.

Essa disparidade é claramente visível nos resultados da política de escalonamento estática com chunk default (Tabela 2 e Tabela 4). Ao particionar a matriz em blocos contíguos de $\sim N/p$ linhas de forma determinística, o OpenMP inevitavelmente destinou

conjuntos massivos de iterações mais custosas computacionalmente a apenas uma ou duas threads que operam no "meio" do fractal. Como consequência empírica, o Fator de Balanceamento de Carga saltou para $\sim 3,00$. As demais threads, ao finalizarem os blocos menos custosos correspondentes às bordas da imagem, permaneceram ociosas à espera das threads centrais, prejudicando a eficiência paralela para insatisfatórios 29% com 16 threads.

Para contornar esse gargalo e redistribuir o trabalho ocioso, políticas iterativas foram testadas, revelando relações críticas entre balanceamento de carga e o overhead de atribuição em tempo de execução:

- **Static com granularidade fina (chunk=1):** Através da estratégia round-robin, a atribuição de pequenos blocos "diluiu" o peso da região central do fractal por entre todas as threads. O escalonamento "*static,1*" ofereceu um fator de balanceamento excelente de $\sim 1,21$ e com a vantagem teórica de gerar zero overhead de sincronização de filas dinâmicas.
- **Dynamic com granularidade fina (chunk=1):** Ao adotar a filosofia de fila sob demanda, o "*dynamic,1*" não apenas resolveu o desbalanceamento espacial do fractal, mas também mitigou anomalias pontuais do sistema operacional, garantindo que as threads só assumissem novos blocos se estivessem efetivamente ociosas. Esse fenômeno é reforçado pelos dados obtidos durante a experimentação. Como evidencia a Seção 3 deste presente relatório, essa configuração mostrou-se consistentemente melhor em relação às outras, alcançando um notável speedup de $\sim 10,5x$ no input padrão e atingindo o balanceamento mais próximo da perfeição na análise de estresse com Fator $\sim 1,004$ (Tabela 6). A granularidade fina linha-a-linha suplantou o overhead adicional exigido por esta política devido à altíssima disparidade da carga.
- **O Guided:** Surpreendentemente, a política *guided*, inicialmente pensada como a que iria extrair o melhor da distribuição estática inicial e do balanço dinâmico final reduzindo o overhead das requisições frequentes das threads à fila principal, registrou o pior desempenho global ao se extrair os melhores chunks (Tabela 3). O bloco inicial calculado dinamicamente pelo algoritmo interno (proporcional às iterações restantes) mostrou-se demasiadamente "espesso". Ao aplicar grandes porções na fase inicial de computação, a política "engoliu" grandes frações custosas de forma contígua antes que a fase de refinamento fino do algoritmo de escalonamento pudesse efetivamente intervir, gerando um Fator consideravelmente alto de $\sim 1,60$.

4.4 Escalabilidade e Limitações Arquiteturais

A análise da escalabilidade forte revela dois comportamentos distintos conforme a política. Para as políticas bem balanceadas, como é no caso de *dynamic* e *static* com

chunk pequeno, o tempo decai de forma aproximadamente proporcional até 4 threads, com eficiência próxima de $\sim 0,97$ o que é quase ideal nessa faixa.

Apesar disso, os testes de escalabilidade evidenciam uma notável degradação no desempenho geral a partir da marca de 8 threads (Gráfico 3). Embora tenhamos empregado 16 threads sob as melhores condições de paralelismo de dados, o speedup estagnou em torno de $\sim 10,5\times$, não acompanhando a curva linear ideal.

O primeiro fator que pode explicar essa limitação é inerente à Lei de Amdahl. Embora a fase computacional seja altamente paralisável, os tempos inerentes de overhead na criação e orquestração do conjunto de threads, das operações intrinsecamente seriais de I/O e da alocação de matrizes restringem o limite teórico superior de aceleração do sistema.

Outro fator extremamente importante para a explicação dessa limitação está no impacto que há na natureza híbrida do processador utilizado. O Intel Core 7 240H possui 10 núcleos físicos, mas 16 CPUs lógicas, por tanto, é uma combinação de núcleos de performance P-Cores com Hyper-Threading e núcleos de eficiência E-Cores sem Hyper-Threading.

O que significa que à medida que o limite de threads escalou de 8 para 16, a implementação passou a utilizar threads lógicas concorrendo pelas mesmas unidades de hardware nos P-Cores, e também precisou utilizar os E-Cores, que operam de forma mais lenta e eficiente em termos energéticos, mas desfavorecem um processamento intensivo em Ponto Flutuante de Dupla Precisão. Em conjunto com a eventual saturação e contenção da largura de banda do Cache L3 compartilhado de 24 MiB, visível na estagnação de tempos com a escalabilidade fraca da Tabela 5 a partir de 8 threads, temos um limite físico prático que impede uma eficiência ideal e que não necessariamente reflete um defeito na paralelização.

4.5 Configuração Mais Consistente

O achado mais relevante desta etapa é que “*dynamic,1*” apresentou consistentemente melhores resultados nos dois cenários, contrariando a intuição teórica inicial de que o guided seria “o melhor dos dois mundos”. No input padrão, “*dynamic,1*” atingiu speedup de $\sim 10,5\times$ enquanto que no vale dos cavalos-marinheiros, onde desbalanceamento acentuado, o speedup foi de $\sim 9,1\times$ tendo sempre o fator de balanceamento próximo de 1,004.

Há duas razões complementares para isso. Primeiro, a granularidade linha-a-linha é o que de fato resolve o problema no Mandelbrot, pois o custo varia de forma acentuada entre linhas vizinhas e, dessa forma, quanto menor a unidade de trabalho melhor é a

"diluição" da região mais custosa. Segundo, o *dynamic* combina essa granularidade fina com adaptação imediata, e aqui os dados revelam um comportamento ainda mais elegante, o *dynamic* compensa inclusive a heterogeneidade do hardware. No teste manual com 16 threads, uma thread processou 271 linhas, enquanto a outra processou apenas 187, em tempos similares de $\sim 0,332$ s. Ou seja, podemos conjecturar que o *dynamic* realiza balanceamento de carga em dois níveis simultâneos, ele dilui o custo heterogêneo dos pixels e ao mesmo tempo adapta a quantidade de trabalho à velocidade de cada núcleo com P-Cores recebendo mais linhas e com E-Cores recebendo menos.

Enquanto isso, o *guided* fracassa exatamente onde deveria se destacar. Seu refinamento progressivo de chunk chega "tarde demais", porque os chunks iniciais espessos já distribuíram de forma irreversível a região cara. A conclusão geral que é importante e deve ser registrada é a de que não existe política universalmente superior, a melhor escolha depende da distribuição espacial do custo. No Mandelbrot, com custo concentrado e variação fina, a granularidade fina em conjunto da adaptação imediata mostrou-se ser a mais adequada para lidar com a natureza do Conjunto de Mandelbrot.

4.6 Escalabilidade Fraca e os Três Regimes

O escalabilidade fraca, onde o trabalho por thread é aproximadamente constante e a resolução vai de 4096^2 a 16384^2 , revelou três regimes distintos de comportamento, todos explicáveis pela arquitetura da máquina utilizada:

- **1 a 4 threads ($\sim 5,6 \rightarrow \sim 6,0$ s):** crescimento quase nulo de +5%. Nessa faixa o problema cabe integralmente nos P-cores, e a escalabilidade fraca é praticamente ideal.
- **4 a 8 threads (salto para $\sim 7,8$ s):** a degradação acentuada marca a entrada em operação dos E-cores e a saturação da hierarquia de memória, ademais, as imagens $8192^2/11585^2$ já não cabem no cache L3 de 24 MiB, passando a depender da largura de banda da DRAM.
- **8 a 16 threads (para $\sim 9,4$ s):** o ganho adicional do Hyper-Threading é marginal, pois as threads irmãs disputam os mesmos recursos de execução.

Esse padrão é típico de uma arquitetura híbrida com cache limitado, e conecta diretamente a escalabilidade fraca à limitação de recursos compartilhados e não ao algoritmo em si, que é perfeitamente paralelizável.

4.7 Outros Gargalos e Limitações

Além das limitações já deliberadas anteriormente nesta presente Seção, outras duas limitações merecem registro:

- **O comportamento não-portável do `schedule(runtime)`:** Verificou-se empiricamente que, com `OMP_SCHEDULE` vazia, o default da libgomp não é static e comporta-se como uma política dinâmica (distribuição desigual de linhas, tempo próximo do melhor explícito). Isso é um comportamento definido pela implementação e, portanto, não-confiável. O que significa que qualquer medição que dependa do default da variável não é reproduzível em outra máquina. A decisão de fixar `OMP_SCHEDULE` explicitamente em todos os experimentos foi, por isso, essencial e é um ponto de transparência metodológica importante.
- **A variância induzida pelo governador de frequência:** O baseline serial oscilou ~25% entre a fase inicial (powersave, ~7,6 s) e a coleta final (performance, ~5,6 s). Isso demonstrou que, sem controle de frequência, os speedups ficariam contaminados por ruído de power management. A fixação do governador em performance foi, portanto, um pré-requisito de confiabilidade, e não um detalhe superficial.

Ademais, por uma questão de rigor, vale registrar que no zoom, o “*dynamic,16*” apresentou tempo médio estatisticamente empatado com o “*dynamic,1*” apresentando uma diferença de ~0,001s, o que está dentro do desvio padrão, muito embora o “*dynamic,1*” tenha o melhor fator de ~1,004. E o “*static,1*” no zoom mostrou um desvio padrão elevado no fator de ~0,108, sugerindo uma instabilidade nessa configuração específica e digna de cautela ao generalizar.

4.8 Sugestões de Melhorias Futuras

Para aprimorar a exploração computacional de cenários fractalizados com OpenMP, a paralelização puramente em laço poderia ser aliada ao paralelismo de tarefas com `#pragma omp task`, subdividindo regiões espaciais complexas do fractal em árvores recursivas ao invés da divisão por linhas, o que concentraria o processamento apenas nas regiões que exigem refinamento intensivo, e assim, o método evitaria varrer completamente as zonas mortas previamente conhecidas do conjunto. Ademais, na etapa MPI, a introdução de uma heurística preditiva de decomposição em blocos, baseada na simetria do próprio Conjunto de Mandelbrot em relação ao Eixo X, eliminaria pela metade a carga computacional global em visualizações que incluam o campo simétrico, duplicando indiretamente os ganhos de speedup.

Ademais, uma varredura sistemática de granularidade (testando chunks de 1, 2, 4, 8, 16, 32 e 64 iterativamente) permitiria traçar a curva exata do trade-off entre a diluição da carga e o overhead da fila, identificando o limite matemático de eficiência de políticas como o guided. Por fim, para isolar cirurgicamente o impacto da arquitetura híbrida (P-Cores vs E-Cores), a futura utilização de afinidade de threads através das variáveis `OMP_PLACES` e `OMP_PROC_BIND` possibilitaria fixar e medir o rendimento isolado de

cada tipo de núcleo, quantificando com exatidão a assimetria do hardware reportada nesta etapa.

5. Conclusão

A Etapa 1 do projeto cumpriu o objetivo proposto: implementar, validar e comparar experimentalmente uma versão sequencial e uma versão paralela em OpenMP do algoritmo de escape-time para geração do Conjunto de Mandelbrot, isolando o efeito do escalonamento de laço sobre o desbalanceamento de carga inerente ao problema. A corretude foi preservada em todos os cenários testados, a matriz de contagens da versão paralela foi bit a bit idêntica à da referência serial, tanto no input padrão quanto no caso de zoom, o que confirma que a paralelização por linha com `schedule(runtime)` não introduziu nenhuma divergência numérica em relação ao algoritmo original.

O principal aprendizado desta etapa não foi que paralelizar reduz o tempo de execução, isso já era esperado, mas sim que a forma como o trabalho é distribuído entre as threads determina o quanto desse potencial é de fato aproveitado. O Conjunto de Mandelbrot evidenciou isso de forma bastante didática: com o particionamento contíguo padrão do static, o fator de balanceamento chegou a $\sim 3,00$ e a eficiência caiu para 0,29 com 16 threads, mesmo com o tempo total caindo, um resultado que, olhado isoladamente, poderia passar a falsa impressão de paralelismo bem-sucedido. Só ao decompor o tempo por thread (Figura 3) ficou claro que as threads das extremidades ficavam praticamente ociosas enquanto as centrais, que herdavam a faixa de maior custo geométrico do conjunto, carregavam sozinhas boa parte do cálculo. Nesse sentido, a política dynamic com chunk unitário se mostrou consistentemente superior nos dois cenários analisados (fator $\approx 1,004$ tanto no input padrão quanto no zoom), o que confirma que, para um problema com custo por pixel tão heterogêneo, o overhead de sincronização por atribuição de chunk é um preço pequeno frente ao ganho de se adaptar em tempo real à carga real de cada thread. Já o guided, apesar de teoricamente projetado para equilibrar overhead e balanceamento, revelou uma fragilidade prática: seu chunk inicial tende a cair justamente sobre a região central mais cara do plano complexo, o que o fez performar pior que o próprio static com o melhor chunk, um resultado contra intuitivo que reforça a importância de validar empiricamente pressupostos teóricos sobre escalonamento, em vez de assumir que a política mais sofisticada é sempre a melhor.

A escalabilidade fraca, por sua vez, expôs os limites do próprio hardware. Mantendo o trabalho por thread aproximadamente constante, o tempo de cálculo deveria, idealmente, permanecer estável, em vez disso, cresceu de forma não linear a partir de 8 threads. Isso evidencia que, numa CPU híbrida com núcleos de performance e de eficiência dividindo uma única cache L3, o balanceamento de carga entre threads é só uma parte do problema, a partir de certo número de threads ativas, largura de banda de

memória e contenção de cache passam a pesar tanto quanto, ou mais do que, a distribuição do trabalho computacional em si.

Do ponto de vista metodológico, expor o schedule via variável de ambiente (schedule(runtime)) foi o que tornou essa comparação viável sem recompilar o binário a cada teste, e a instrumentação por thread (tempos_thread, linhas_thread) foi o que permitiu ir além do tempo total e efetivamente diagnosticar onde o desbalanceamento acontecia, sem essa granularidade de medição, teria sido fácil concluir apenas que "static é mais rápido que guided" sem entender o porquê. As otimizações da branch otimizado (early-exit por cardioide/bulbo, detecção de ciclo por Brent e exploração de simetria do eixo real) seguem a mesma lógica: reduzem o trabalho redundante sem alterar o resultado produzido por pixel, como comprovado pela validação bit a bit, e por isso não substituem uma política de escalonamento adequada, apenas diminuem, para qualquer política escolhida, a quantidade total de trabalho que precisa ser distribuída.

6. Referências

- UESC, Universidade Estadual de Santa Cruz. Notas de aula e enunciado da disciplina DEC107, Processamento Paralelo, 2026.2.
- ETVORELLANA. RepoDEC107-PP-2026_2, Exemplos e exercícios da disciplina DEC107-PP. GitHub, 2026. Disponível em: https://github.com/etvorellana/RepoDEC107-PP-2026_2.
- OPENMP ARCHITECTURE REVIEW BOARD. OpenMP Application Programming Interface, Specification. Disponível em: <https://www.openmp.org/specifications/>.
- BRENT, R. P. An improved Monte Carlo factorization algorithm. BIT Numerical Mathematics, v. 20, n. 2, p. 176–184, 1980. , algoritmo de detecção de ciclo usado na otimização de escape_time (branch otimizado) para identificar órbitas periódicas sem consumir o restante das iterações.
- WIKIPEDIA. Mandelbrot set. Seção "Optimizations", teste analítico de pertencimento à cardioide principal e ao bulbo de período 2. Disponível em: https://en.wikipedia.org/wiki/Mandelbrot_set.
- PEITGEN, H.-O.; JÜRGENS, H.; SAUPE, D. Chaos and Fractals: New Frontiers of Science. 2. ed. New York: Springer, 2004. , referência de apoio aos conceitos teóricos do Conjunto de Mandelbrot e do método de escape-time apresentados na Introdução.

DECLARAÇÃO DE TRANSPARÊNCIA E USO DE INTELIGÊNCIA ARTIFICIAL

Eu, Lucas Braga Orihuela e Cibelle Sousa Rodrigues, regularmente matriculado(a) no curso de Ciência da Computação, declaro para os devidos fins que, na elaboração do trabalho/projeto intitulado "Benchmark de Geração do Conjunto de Mandelbrot (Escape-Time): Etapa 1 — OpenMP", submetido como requisito para a disciplina/atividade DEC107 — Processamento Paralelo, adotei as seguintes práticas em relação ao uso de ferramentas de Inteligência Artificial (IA) generativa:

(Assinale a opção aplicável)

() Não utilizei ferramentas de Inteligência Artificial na concepção, desenvolvimento, programação ou redação deste trabalho.

(X) Utilizei ferramentas de Inteligência Artificial (ex: ChatGPT, GitHub Copilot, Claude, etc.) como recursos de apoio.

Detalhamento do Uso: As ferramentas utilizadas foram Claude (Anthropic), Gemini (Google) e DeepSeek (Hangzhou DeepSeek Artificial Intelligence Basic Technology Research Co., Ltd). O uso se deu exclusivamente nas seguintes etapas do projeto:

- Sugestão de estratégias de otimização de desempenho para a versão OpenMP (Gemini), posteriormente implementadas e testadas pela dupla.
- Apoio na depuração (debugging) do código-fonte quando necessário (Claude).
- Geração do README do repositório e apoio na revisão/comentário do código-fonte (Claude).
- Revisão dos resultados apresentados na Seção 3 com relação aos experimentos feitos na máquina (Gemini e DeepSeek).
- Apoio no entendimento dos resultados obtidos na Seção 3 (DeepSeek).
- Apoio na produção de scripts para automatização dos experimentos (DeepSeek).

Termo de Responsabilidade: Declaro estar ciente de que ferramentas de IA não são isentas de erros, vieses ou alucinações. Sendo assim, atesto que toda a responsabilidade pela autoria, integridade, originalidade e exatidão do conteúdo submetido é exclusivamente minha. Confirmando que todo o código-fonte, dados de testes e textos gerados ou modificados com o auxílio de IA foram rigorosamente revisados, testados e validados por mim, garantindo que os resultados refletem meu próprio entendimento e esforço intelectual, sem incorrer em plágio intelectual ou acadêmico.

Ilhéus - BA, 21/09/2026

Lucas Braga Orihuela
Matrícula: 202311125

Cibelle Sousa Rodrigues
Matrícula: 202310768